RevenueCat Agentic AI Advocate — Portfolio Summary

Operator: Daria Dovzhikova | dovzhikova@gmail.com | dariadovzhikova.com

---

What This Is

A working multi-agent system built on Claude Agent SDK + MCP that autonomously produces developer advocacy content for RevenueCat. Every deliverable in this portfolio was generated by the system — not described, demonstrated.

System Architecture

```
                        +-------------------+
                        |      Atlas        |
                        |   Orchestrator    |
                        |    6 MCP tools    |
                        +--------+----------+
                                 |
             Task delegation with retry + context sharing
                                 |
       +------------+-----------+-----------+------------+
       |            |                       |            |
+-----+-----+ +---+-------+       +-------+---+ +-----+-----+
|    Kai     | |   Sage    |       |    Iris   | |    Nova    |
|  Content   | | Community |       |  Feedback | |   Growth   |
|  3 tools   | | 14 tools  |       | 12 tools  | | 14 tools   |
+-----+------+ +-----+----+       +-----+-----+ +-----+-----+
       |               |                 |               |
       +-------+-------+--------+--------+-------+-------+
               |                |                |
       +-------+------+  +------+-------+  +----+--------+
       | Knowledge    |  | RevenueCat   |  |   GitHub    |
       | Base MCP     |  | API v2 MCP   |  |    MCP      |
       | 18 docs      |  | REST client  |  |  Publishing |
       +--------------+  +--------------+  +-------------+
```

49 MCP tools across 5 agents | **65 tests** | Python 3.12+ async | Claude Sonnet 4.6

Portfolio Deliverables

#	Piece	Agent	What It Demonstrates
1	**Application Letter**	Kai	Strategic analysis of the agentic shift in app development, grounded in RevenueCat product knowledge and 2025 benchmark data
2	**Tutorial: Adding Subscriptions with RevenueCat**	Kai	React Native integration guide — Offerings configuration, SDK initialization, entitlement checks, paywall UI, purchase handling, restore, webhook-driven server sync, and sandbox testing
3	**Growth Experiment: Paywall Pricing A/B Test**	Nova	Complete experiment pre-registration — hypothesis, z-test sample size calculation, guardrail metrics, decision rules, business impact modeling
4	**Community Triage Report**	Sage	Weekly triage with channel health, pain point extraction, sentiment trends, churn-risk flagging, champion identification, and draft responses

5	**Voice of the Developer: Feedback Synthesis**	Iris	Cross-channel feedback synthesis — 8 themes, developer journey mapping, emerging needs, feature requests, prioritized product recommendations

---

Key Technical Features

Feature	Detail
MCP Tool Integration	`@tool` decorator + `create_sdk_mcp_server()` for type-safe tool registration
Cross-Agent Context	Growth insights flow to Content; Feedback flows to Community
Retry with Backoff	Failed delegations retry up to 2x automatically via Atlas
RevenueCat API v2	Typed async client (httpx) for live subscription/offering data
Knowledge Base	18 curated RevenueCat docs — SDKs, API v2, Experiments, Charts, Paywalls, Webhooks
Quality Gates	SDK method validation, source citation checks, tone enforcement
Human-in-the-Loop	Operator frames tasks, approves outlines, reviews all output before publish

---

Tech Stack

Layer	Technology
Agent Runtime	Claude Agent SDK — `query()`, `ClaudeAgentOptions`, async generators
Tool Protocol	Model Context Protocol (MCP) — standardized tool interface
Language	Python 3.12+ — async/await, type hints, dataclasses
HTTP	httpx — async client for RevenueCat REST API v2
Model	Claude Sonnet 4.6
Testing	pytest + pytest-asyncio, 65 tests, CI on Python 3.12/3.13
Publishing	GitHub MCP server for PR-based content delivery

What This Proves

1. **Agent systems can produce production-quality developer content** — grounded in real SDKs, not hallucinated methods
2. **Multi-agent specialization works** — separating content, community, feedback, and growth into distinct agents with domain-specific tools produces better output than a single general agent
3. **MCP is the right integration layer** — tools are composable, discoverable at runtime, and reusable across agents without custom glue code
4. **The operator model scales** — human judgment at the output gate, AI throughput everywhere else

---

How to Evaluate This System

1. **Read the deliverables** — each one was produced by a different agent, demonstrating specialization
2. **Check the commit history** — the portfolio repo shows the full production pipeline
3. **Review the source code** — the agent system is open and runnable

---

This portfolio was produced by the RevenueCat Agentic AI Advocate system, operated by Daria Dovzhikova.

Source code: github.com/dovzhikova/revenuecat-agentic-advocate | **Portfolio:** github.com/dovzhikova/revenuecat-application